

Chronos

A Content-Addressed Versioned Data Store

A Git-Inspired Approach to Structured Data Versioning Using Merkle Search Trees

WORKING DRAFT · ARCHITECTURE AND METHODOLOGY COMPLETE · FINAL RESULTS PENDING

Chronos Project Team
23 August 2026

Draft status

Component	Status	What this draft contains
Architecture	Complete	System boundary, object model, persistence, and workflows.
Methodology	Complete	Variants, workloads, metrics, fairness, and reproducibility protocol.
Literature review	Working complete	Nine-source matrix and synthesis; supervisor review still recommended.
Results	Preliminary only	Imported solely from the latest smoke benchmark CSV export.
Final claims	Gated	Require paper-profile runs with Dolt installed and reviewed CSVs.

Evidence rule. No performance value in this document was invented or transcribed from memory. The preliminary table and chart are generated from output/benchmarks/check-again/summary.csv.

Abstract

Versioning structured datasets requires a balance between cheap writes, compact storage, rapid historical reconstruction, and efficient comparison. Chronos is a Python research prototype that stores immutable state in a persistent fixed-depth Merkle hash trie. Each commit rewrites only the affected leaf buckets and their ancestor paths, while unchanged subtrees are reused by content hash. Chronos-H augments this tree with content-addressed changesets and adaptively selects either operation-log aggregation or hash-pruned tree comparison for a version diff. SQLite provides atomic persistence for addressed objects, version aliases, measurements, and HEAD without defining the indexing semantics. This paper specifies the architecture and an experimental methodology comparing Snapshot, Log-only, forced-Merkle Chronos-M, Chronos-H, and Dolt. A preliminary 1,000-row smoke run validates correctness and the measurement pipeline, but Dolt was unavailable; consequently, this draft makes no state-of-the-art performance claim. Final conclusions are reserved for repeated paper-profile experiments using identical deterministic workloads.

Keywords—content-addressed storage; dataset versioning; Merkle trie; copy-on-write; adaptive differencing; SQLite; Dolt

1. Introduction

Dataset versioning is not merely file archival. A useful system must preserve exact history while supporting incremental updates, historical checkout, and change inspection without repeatedly scanning or copying every record. Git demonstrates how immutable, content-addressed objects and commit graphs make history reproducible [2]. Structured data introduces different access patterns: records have stable keys, updates are often sparse, and diffs may span either a few nearby commits or a large history distance.

Chronos tests a deliberately narrow research hypothesis: whether a fixed-depth, hash-routed Merkle trie with incremental path rewriting can be combined with an operation index so that the system chooses an efficient diff path for the observed history distance. It is not presented as a SQL database or a replacement for Dolt. Dolt is the state-of-the-art production comparator because it combines SQL semantics with Git-style history over Prolly Trees [3]. Chronos instead exposes a smaller, instrumentable design whose internal work can be measured directly.

1.1 Research questions

1. RQ1: Does incremental copy-on-write hashing reduce work relative to full-state snapshotting as datasets grow?
2. RQ2: When does changeset aggregation outperform hash-pruned Merkle comparison, and can a calibrated selector capture that boundary?

3. RQ3: What latency, storage, memory, and examined-work trade-offs does Chronos-H exhibit against Snapshot, Log-only, Chronos-M, and Dolt?

1.2 Claimed contributions

- A deterministic fixed-depth Merkle hash trie with canonical SHA-256 addressing and incremental multi-key path rewriting.
- A hybrid diff policy combining explicit changesets for short ancestor paths with hash-pruned tree comparison for larger histories.
- A SQLite durability layer that atomically commits immutable objects, metrics, version aliases, and HEAD while verifying hashes on reopen.
- A reproducible harness that runs shared workloads across four implemented variants and a native Dolt adapter, exporting raw and summarized CSV evidence.

2. Literature Review

A literature review maps what prior systems already solve, how they solve it, and which unresolved design question motivates the present work. For Chronos, the relevant line runs from authenticated hash structures and Git object identity to database-native branching, dataset-version storage trade-offs, and modern content-addressed structured stores.

2.1 Foundations

Merkle's authenticated tree construction established the principle that a compact root digest can commit to a much larger collection [1]. Git operationalizes content-addressed objects, trees, and commits in a developer-facing version-control model [2]. Chronos borrows immutable object identity and parent-linked commits, but routes structured keys through a fixed-depth trie rather than representing a directory tree.

2.2 Dataset and relational versioning

DataHub framed collaborative dataset management as a first-class platform problem [4]. Decibel integrated branching into a relational storage engine and evaluated version-first, tuple-first, and hybrid layouts [5]. OrpheusDB instead provided bolt-on relational versioning and emphasized the storage-versus-recreation trade-off [6], a trade-off formalized more generally by dataset-versioning research [7]. These systems motivate Chronos's explicit measurement of commit, checkout, storage, and diff costs rather than a single throughput metric.

2.3 Content-addressed structured stores

Noms organizes typed data into content-addressed chunks [8], while ForkBase offers immutable, tamper-evident storage and fork semantics for blockchain and forkable applications [9]. Dolt uses a Git-style commit graph whose table state is represented by Prolly Trees, content-addressed B-tree-like structures designed for structural sharing and efficient diff [3]. Chronos differs by fixing trie depth and exposing both operation- and tree-based diff work. This makes the model simpler to instrument, but it also sacrifices Dolt's ordered range behavior, SQL surface, mature branching, merge, and production engineering.

2.4 Literature-review matrix

Work	Representation	Key contribution	Gap / relationship to Chronos
Merkle [1]	Authenticated hash tree	Root digest commits to a set	Foundation; not a data-versioning system
Git [2]	Content-addressed objects + commit DAG	Trees, parents, object reuse	File/tree oriented; no structured query model
Dolt [3]	Git-style history over Prolly Trees	Ordered structural sharing; native diff	Production SOTA comparator; much broader SQL scope
DataHub [4]	Collaborative dataset platform	Dataset history and workflows	Platform vision rather than Chronos's tree/diff mechanism
Decibel [5]	Relational branching engine	Version-/tuple-first and hybrid layouts	Built-in relational engine; different storage model
OrpheusDB [6]	Bolt-on relational versioning	Partitioned version storage/retrieval	Focuses version recreation, not Merkle pruning
Dataset versioning [7]	Version graph optimization	Formal storage/recreation trade-off	Analytical basis; not an executable adaptive diff
Noms [8]	Typed content-addressed chunks	Decentralized immutable data	Broader data model; Chronos evaluates fixed trie + logs
ForkBase [9]	Forkable immutable storage engine	Content addressing and fork semantics	Distributed/forking scope exceeds current Chronos

Matrix entries are analytical summaries of primary papers or official project documentation; full citations appear in Section 9.

3. Chronos Architecture

Chronos is separated into interface, service, model, and durability layers. The research contribution is the versioned storage model and its instrumented diff policy; the React frontend and REST API make the prototype demonstrable but are not treated as algorithmic contributions.

Chronos layered architecture

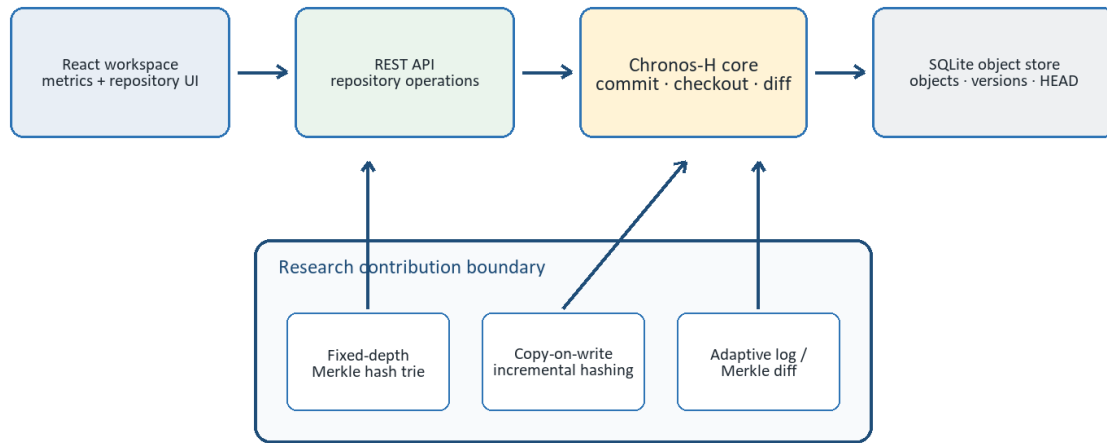


Figure 1. Layered Chronos architecture and the research contribution boundary.

3.1 Content-addressed object model

Object	Identity / reference	Purpose
Trie node	SHA-256 of canonical node JSON	Internal child hashes or a canonical leaf bucket.
Changeset	SHA-256 of canonical operation list	Records the exact batch used by log-based diff.
Commit	SHA-256 over root, parent, changeset, message, timestamp	Immutable version identity and linear history link.
Version alias	Sequential integer v1, v2, ...	Human-readable pointer to a commit; not the content identity.
HEAD	Named SQLite reference	Points to the latest durable commit.

3.2 Fixed-depth Merkle hash trie

The default trie has branching factor $b=8$ and depth $d=4$. SHA-256(key) supplies three routing bits per level, selecting one of 4,096 possible leaf buckets after twelve bits. Full keys remain in the leaf, so routing-prefix collisions preserve correctness. Under uniform routing, expected leaf occupancy is approximately N / b^d ; at 100,000 records this is about 24.4 records per bucket. The value is a tunable design point, not a universal optimum, and the final study must sweep depth and branching factor.

3.3 Incremental hashing

A commit does not rebuild the tree. Changed keys are routed and grouped by common prefixes; every affected leaf is rewritten once, and only the union of affected ancestor paths is rebuilt. Unchanged child

hashes are copied into the new internal nodes, preserving structural sharing. For a single changed key, the model creates roughly $d+1$ trie nodes; batched keys sharing a prefix also share rewritten ancestors.

3.4 Commit and diff workflow

Incremental commit and adaptive diff workflow

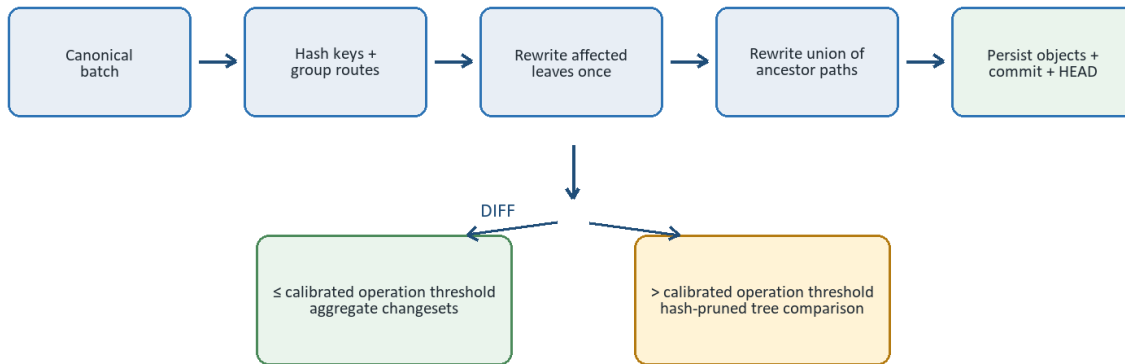


Figure 2. Incremental commit path and Chronos-H adaptive diff decision.

3.5 Diff algorithms

Mode	Procedure	Measured work	Expected strength
Log	Walk the ancestor path and aggregate key operations.	Operations examined	Short history distance or few accumulated operations.
Merkle	Compare roots recursively; skip equal subtree hashes.	Node pairs + leaf entries	Large histories with substantial unchanged structure.
Hybrid	Count ancestor operations; choose log at or below threshold, otherwise Merkle.	Selected mode's unit	Avoid a single diff mechanism across all regimes.

The current threshold is calibrated independently of evaluation workloads. In the latest smoke export, the selected threshold was 128 operations. Because the calibration points ended at 128 operations and log diff was still faster there, this value is a policy boundary derived from the sampled grid—not proof of a true performance crossover. The final paper should extend the calibration range.

3.6 SQLite durability boundary

SQLite stores, but does not compute, Chronos semantics. The schema contains metadata, addressed objects, version-to-commit mappings, commit statistics, and refs. A BEGIN IMMEDIATE transaction inserts new objects, the version and metrics, and the updated HEAD atomically. On reopen, Chronos verifies canonical payloads, object hashes, child and commit references, contiguous versions, the parent

chain, metrics, and HEAD. Failed persistence reloads the last durable in-memory model so volatile state cannot advance beyond disk.

3.7 Current scope

- Single linear HEAD; named branches, merge commits, and conflict resolution are not implemented.
- One serialized writer; SQLite is used as an embedded object container, not as the versioning algorithm.
- Values are stored inline in leaf nodes; blob chunking and garbage collection are not implemented.
- The API supports repository creation/opening, CSV import, batch commits, history, checkout, comparison, and metrics.

4. Methodology

4.1 Experimental variants

Variant	Durable representation	Diff mechanism	Role
Snapshot	Canonical full-state JSON/version	Full key scan	Storage/time lower-bound baseline for simple implementation.
Log-only	Base state + fsynced JSONL changesets	Replay / aggregate operations	Operation-log baseline.
Chronos-M	SQLite fixed-depth Merkle trie	Forced hash-pruned tree diff	Ablation isolating adaptive selection.
Chronos-H	Same trie + addressed changesets	Calibrated log/Merkle selection	Primary proposed system.
Dolt	Native repository and keyed SQL table	Native dolt diff	Production SOTA comparator.

Chronos-M is an ablation, not a separate product claim. The main comparison remains Snapshot, Log-only, Chronos-H, and Dolt; Chronos-M answers whether adaptive selection changes the forced-Merkle design.

4.2 Workloads

Each scenario is generated once from a fixed seed and reused byte-for-byte across adapters and repetitions. Initial keys are deterministic, values are deterministic SHA-256-derived payloads, and each batch contains updates with insert/delete operations when the batch size permits. Spread and hot-key locality are tested separately. The current paper profile is:

Scenario	Rows	Commits	Changes/commit	Locality
small-sparse	1,000	10	10	spread

Scenario	Rows	Commits	Changes/commit	Locality
medium-sparse	10,000	10	10	spread
medium-dense	10,000	10	1,000	spread
large-sparse	100,000	10	100	spread
large-hot	100,000	10	100	hot

A 1,000,000-row stress point may be added if the machine completes it reliably; it must be reported as an extension, not silently mixed into the preset profile.

4.3 Calibration and evaluation separation

Calibration sweeps operation counts using workloads that are not reused for final evaluation. The selector chooses the largest tested count at which median log diff is no slower than median Merkle diff. The frozen threshold is then used for all Chronos-H evaluation trials. This prevents selecting the policy after observing the headline workloads.

4.4 Metrics and measurement contract

Metric	Operational definition
Initial import	Time to create the first durable state and version; repository setup excluded.
Incremental commit	Median duration of durable commits within one trial.
Diff	Time to consume a complete comparison from the initial to final version.
Checkout	Time to fully materialize the target historical version.
Storage	Total regular-file bytes inside the fresh adapter repository directory.
Peak memory	Python allocations via tracemalloc in current exports; Dolt RSS only if externally sampled.
Work examined	Natural unit: keys, operations, log operations, or trie-node pairs; units are not interchangeable.
Correctness	Exact initial/final checkout and changed-key oracle where structured keys are exposed.

4.5 Repetition and reporting

- Run on an idle, AC-powered machine using fresh repositories for every trial.
- Use two warmups and seven measured paper-profile trials; exclude warmups from summaries.
- Report median, 25th percentile, and 75th percentile rather than only the fastest run.
- Record seed, workload digest, platform, Python version, Dolt version, selected strategy, and metric method in every export.
- Retain raw_results.csv, summary.csv, and manifest.json together; treat the manifest as part of the evidence.

4.6 Fairness and reproducibility

The same logical states and mutation batches are presented to every adapter. Chronos and baselines use native durable representations; Dolt uses a keyed SQL table, commits/tags each version, checks history with AS OF, and invokes native dolt diff. This is a repeatable CLI-facing workflow, not an isolated engine microbenchmark. Python tracemalloc and external process RSS are different memory quantities and must not appear on one comparative chart unless all variants are rerun under the same external monitor. Dolt exposes no comparable internal node counter, so work-examined remains blank.

4.7 Analysis plan

1. Verify correctness and workload digests before interpreting speed.
2. Plot median and interquartile range against rows, mutation density, history distance, and locality.
3. Compare Chronos-H with Chronos-M to isolate the selector; compare both with simple baselines to expose overheads.
4. Compare Chronos-H with Dolt conservatively, reporting where it wins, ties, or loses and separating prototype scope from SQL capability.
5. Repeat any anomalous cell and preserve both the original and confirmation exports.

5. Preliminary Results: Pipeline Validation Only

NOT FINAL PAPER EVIDENCE · SMOKE PROFILE · DOLT UNAVAILABLE

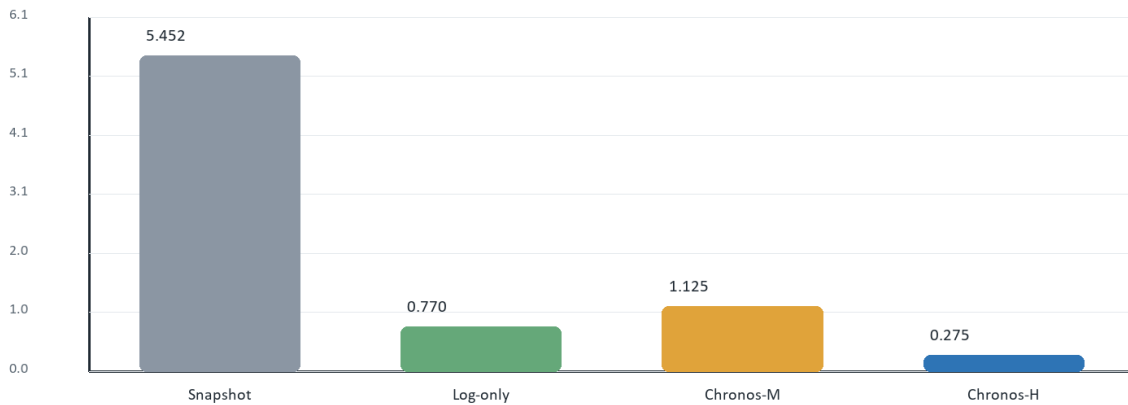
The latest reviewed export (2026-08-22T17:05:17.849258+00:00) used Python 3.14.3 on Windows-11-10.0.26200-SP0, one warmup, and 3 measured trials. The evaluation scenario contained 1,000 rows, four commits, and ten changes per commit with spread locality. All four available implementations passed the correctness oracle. Dolt produced no timing rows because it was not installed.

Model	Import ms	Commit ms	Diff ms	Checkout ms	Storage KiB	Diff path
Snapshot	14.27	13.79	5.452	2.79	249.0	full-scan
Log-only	9.56	3.86	0.770	3.19	55.2	operation-log
Chronos-M	221.34	25.28	1.125	1.22	584.0	merkle
Chronos-H	246.93	29.48	0.275	5.96	580.0	log

Source: output/benchmarks/check-again/summary.csv; medians over three measured trials. Values are reproduced programmatically from the CSV.

Preliminary smoke diff latency (median, ms)

1,000 rows · 4 commits · 10 changes/commit · n=3 measured trials



Source: output/benchmarks/check-again/summary.csv. Dolt absent; not a final comparison.

Figure 3. Preliminary smoke-profile diff latency. The chart is diagnostic, not a Dolt comparison.

5.1 What the smoke run supports

- The end-to-end harness, CSV summarization, correctness oracle, and four implemented adapters execute successfully.
- Chronos-H selected log for 40 accumulated operations and recorded a 0.275 ms median diff, versus 1.125 ms for forced Merkle.

- On this single small scenario, Chronos-H diff was 19.9× lower than Snapshot; this ratio must not be generalized beyond the smoke workload.
- The Chronos variants incurred materially higher initial-import latency and storage than the simple baselines, demonstrating a real prototype overhead that the final discussion must retain.

5.2 What the smoke run does not support

- No claim that Chronos is faster, smaller, or more scalable than Dolt.
- No claim that 128 operations is the true log/Merkle crossover; the calibration grid did not sample beyond 128 in this run.
- No asymptotic conclusion from one dataset size, one locality pattern, four commits, and three trials.
- No fair cross-process memory comparison because the current Python and Dolt measurement methods differ.

6. Discussion

6.1 Interpretation of the design

Chronos's value is not that it reproduces Dolt with fewer features. Its research value lies in making three costs explicit: incremental structural rewrite, log aggregation, and hash-pruned tree traversal. A fixed-depth trie gives predictable routing and a compact implementation, while a separate changeset index gives the system a second diff path. The ablation determines whether that second path improves the Merkle-only model.

The smoke results are consistent with the motivation: when only 40 operations separate versions, aggregating those operations examines less work than traversing 362 trie-node pairs. The same export also shows the cost of the design: Chronos initial import and storage exceed the Snapshot and Log-only baselines at 1,000 rows. A credible final paper must present both sides and identify the regions in which structural sharing repays its metadata overhead.

6.2 Relationship to Dolt

Dimension	Chronos-H	Dolt
Core structure	Fixed-depth Merkle hash trie + changesets	Prolly Trees in a Git-style commit graph
Interface	Key/value repository, REST API, research UI	SQL database, CLI, branches, merges, remotes
Diff	Instrumented adaptive log/Merkle selection	Native structural diff over production engine
Purpose	Controlled prototype for mechanism-level evaluation	General production database and collaboration system

Dimension	Chronos-H	Dolt
Current maturity	Linear history; Python; single writer	Mature cross-platform SQL product

Therefore, absolute latency is only one axis. The paper must avoid presenting feature disparity as an algorithmic advantage: Dolt performs SQL parsing, schema management, and production durability work outside Chronos's current scope. The strongest defensible result would identify specific workload regions where Chronos-H's simplified mechanism reduces measured diff work while openly reporting its missing capabilities and overheads.

6.3 Decision rules for final claims

- Use 'outperformed on scenario X' only when the reviewed median and IQR support it; avoid an unqualified 'faster than Dolt.'
- Treat Chronos-M versus Chronos-H as the mechanism ablation; do not inflate the headline comparison with unnecessary variants.
- Report losses and ties with the same prominence as wins.
- Separate statistical observations from architectural explanations; label explanations as hypotheses unless directly measured.

7. Limitations and Threats to Validity

Threat	Why it matters	Mitigation / reporting rule
Language/runtime asymmetry	Chronos is Python; Dolt is a mature Go product.	Frame as system-level comparison and add work counters where available.
Feature asymmetry	Dolt includes SQL, branches, merges, and production features.	Do not convert lower prototype overhead into a universal superiority claim.
Fixed trie parameters	b=8, d=4 may favor some sizes/distributions.	Run a branching-factor/depth sensitivity sweep.
Linear history	Current log diff assumes ancestor traversal and no merges.	State scope; future work adds named branches and merge-aware diff.
Work-unit mismatch	Keys, operations, and node pairs are different quantities.	Plot within-unit trends; never treat them as one normalized count.
Memory instrumentation	tracemalloc omits non-Python allocations; Dolt needs RSS.	Rerun all systems under one external sampler before comparison.
OS and cache effects	Windows scheduling and filesystem cache affect timings.	Idle machine, fresh repositories, warmups, repeated trials, median/IQR.
Synthetic data	Uniform hashed keys may not capture real skew or payload diversity.	Add at least one public structured dataset and a declared key mapping.
Smoke evidence	Current results cover one small workload and no Dolt.	Keep as pipeline validation; replace tables after paper-profile export review.

8. Completion Plan and Provisional Conclusion

8.1 Evidence gate before submission

1. Install a pinned Dolt release, confirm dolt version is recorded, and run the adapter correctness path independently.
2. Extend threshold calibration beyond 128 operations; freeze the selected threshold before evaluation.
3. Run the paper profile with two warmups and seven measured trials on an idle machine.
4. Review `raw_results.csv` for failures/outliers, verify every workload SHA-256, and regenerate `summary.csv` without manual edits.
5. Run the trie parameter sweep and, if feasible, the declared 1,000,000-row stress extension.
6. Replace Section 5 with final tables/plots; then finalize results, discussion, limitations, abstract, and conclusion.
7. Archive the exact harness commit, Dolt version, manifests, reviewed CSVs, and plotting script used for the submitted paper.

8.2 Provisional conclusion

Chronos demonstrates a coherent architecture for content-addressed structured-data versioning: immutable canonical objects, incremental copy-on-write Merkle trie updates, durable commits in SQLite, and a diff path that can select between changeset aggregation and structural hash pruning. The smoke experiment confirms functional correctness and the evidence pipeline, and it illustrates why adaptive selection is worth evaluating. It does not yet establish comparative performance against Dolt. The final contribution will depend on the reviewed paper-profile results and on a restrained account of where the hybrid design helps, where it costs more, and how its limited prototype scope differs from a production SQL database.

9. References

- [1] R. C. Merkle, “A Digital Signature Based on a Conventional Encryption Function,” *Advances in Cryptology—CRYPTO ’87*, LNCS 293, pp. 369–378, 1988. https://doi.org/10.1007/3-540-48184-2_32
- [2] S. Chacon and B. Straub, “Git Internals—Git Objects,” *Pro Git*, 2nd ed. <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>
- [3] DoltHub, “Dolt Storage Engine Architecture.” <https://www.dolthub.com/docs/architecture/storage-engine/>
- [4] A. Bhardwaj et al., “DataHub: Collaborative Data Science & Dataset Version Management at Scale,” *arXiv:1409.0798*, 2014. <https://arxiv.org/abs/1409.0798>
- [5] A. Maddox et al., “Decibel: The Relational Dataset Branching System,” *Proceedings of the VLDB Endowment*, vol. 9, no. 9, pp. 624–635, 2016. <https://doi.org/10.14778/2947618.2947619>
- [6] S. Huang et al., “OrpheusDB: Bolt-on Versioning for Relational Databases,” *Proceedings of the VLDB Endowment*, vol. 10, no. 10, pp. 1130–1141, 2017. <https://arxiv.org/abs/1703.02475>
- [7] S. Bhattacharjee et al., “Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff,” *Proceedings of the VLDB Endowment*, 2015. <https://pmc.ncbi.nlm.nih.gov/articles/PMC5526644/>
- [8] Attic Labs, “Noms: Introduction.” <https://github.com/attic-labs/noms/blob/master/doc/intro.md>
- [9] S. Wang et al., “ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications,” *Proceedings of the VLDB Endowment*, vol. 11, no. 10, pp. 1137–1150, 2018. <https://www.vldb.org/pvldb/vol11/p1137-wang.pdf>

Appendix A. Reproducible Commands

Smoke verification

```
python -m experiments.final_benchmark --profile smoke --warmups 1 --trials 3
```

Final paper run

```
python -m experiments.final_benchmark --profile paper --warmups 2 --trials 7
```

Run discipline. Do not edit benchmark CSVs manually. Preserve raw results, summary, and manifest as one evidence bundle.